Lab #01-01: Configuring iptables Rules Manually and Using Fwbuilder

Student Name:	**Mohammad Salem Hassani**
Course:	Cybersecurity / Network Defense
Instructor:	Asen Grozdanov
Due Date:	November 29th, 2025, Midnight (23:59)
Submission Platform:	Omnivox Lea (DropBox)
GitHub Repository:	github.com/salem228h/Cybersecurity-Portfolio
GitHub Username:	salem228h
Lab Date:	November 2025

1. Lab Overview

- This lab demonstrates hands-on experience with Linux firewall configuration using iptables and Fwbuilder. The primary objectives are:

  • Understanding and implementing iptables rules manually
  • Configuring firewall policies to control network traffic
  • Testing and validating firewall rules
  • Using Fwbuilder as a graphical interface for advanced firewall management
  • Documenting the configuration process with screenshots and explanations

  This lab is part of the Network Defense/Cybersecurity course curriculum and demonstrates practical skills in network security and system administration.

Lab Environment Reference

Network Configuration:

- Subnet: 10.0.2.0/24
- Router VM: Xubuntu Router
  - ✓ WAN (Internet): enp0s3
  - ✓ LAN (Gateway): enp0s8
  - ✓ IP Address: 10.0.2.1
- Client VM: Client Machine
  - ✓ Interface: eth0
  - ✓ Gateway: 10.0.2.1
  - ✓ IP Address: 10.0.2.100

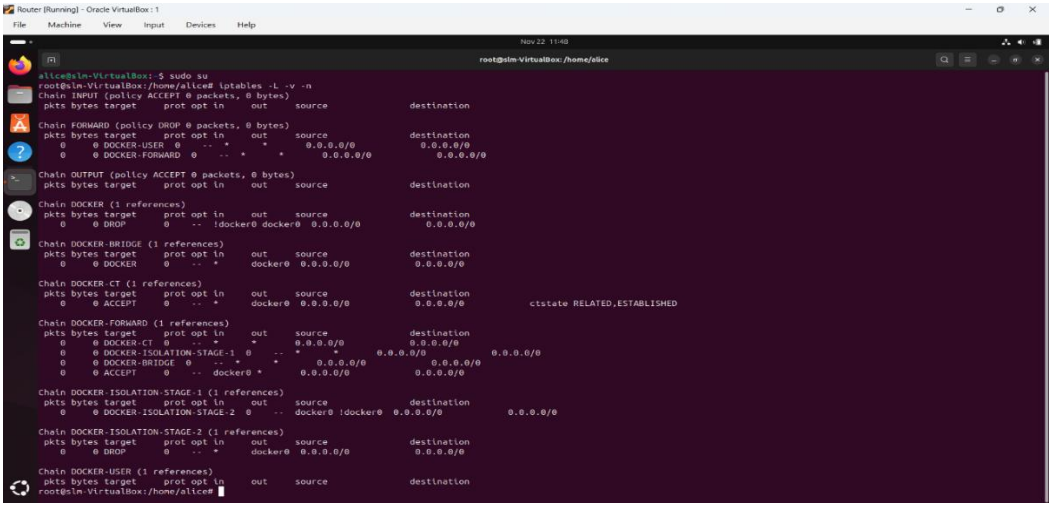
Stage 1: Setup and Initial Screenshots

Objective: Prepare the Router VM and capture initial system state.

Tasks to Complete:

1. **Login to Router VM** (Xubuntu Router)
2. **Open Terminal**
3. **Switch to root user:**

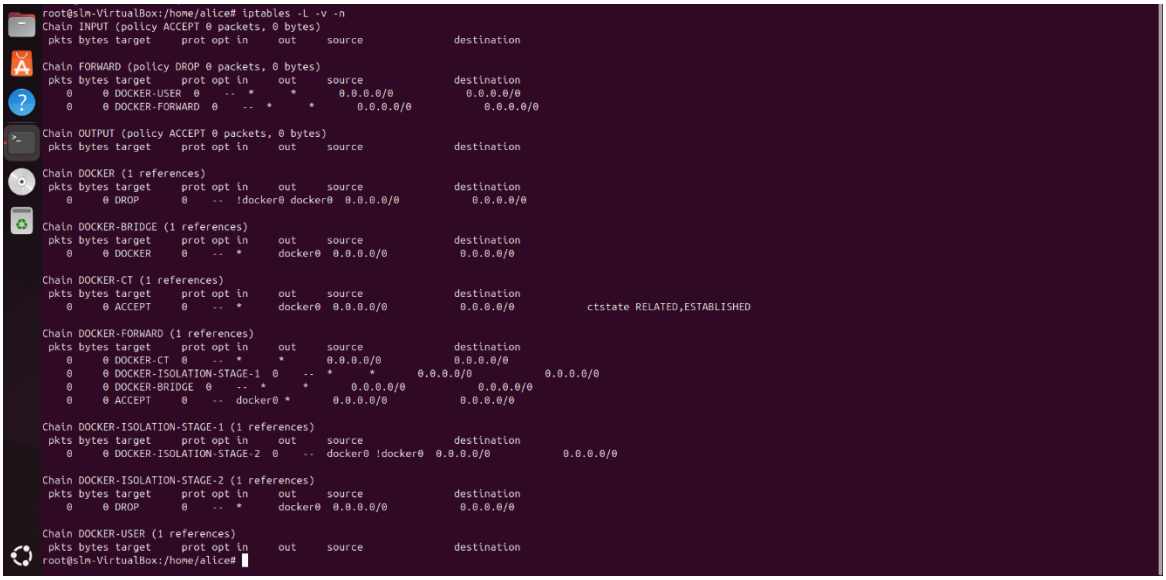
   ## sudo su



Terminal showing root access

4.Check initial iptables status:

iptables -L -v -n



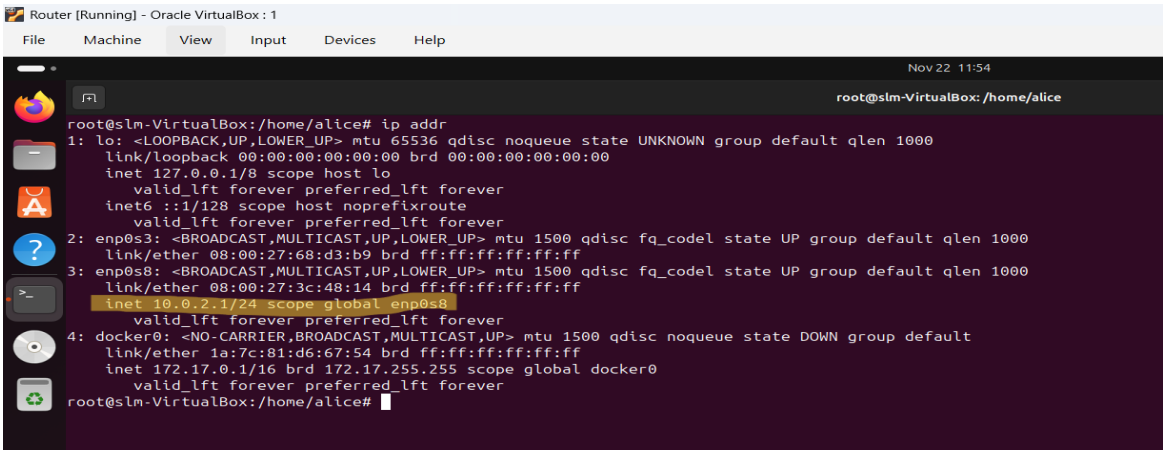
Complete output showing empty firewall

5.Verify IP addresses and interfaces:

ip addr

Expected output:

- enp0s3 → WAN interface
- enp0s8 → LAN interface
- IP on enp0s8 = 10.0.2.1



Complete output showing interfaces and IP addresses

PART 1: THE IPTABLES COMMAND

Stage 2: Challenge #1 - Default Policies

Objective: Set default POLICY to ACCEPT on INPUT and OUTPUT chains, and DROP on FORWARD chain.

Commands:

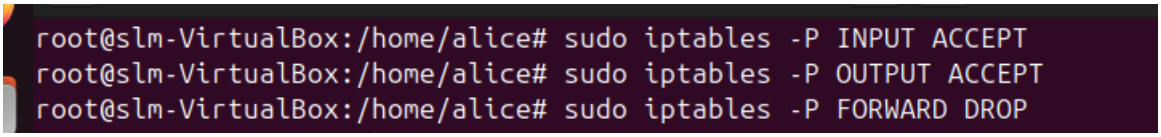
sudo iptables -P INPUT ACCEPT

---

sudo iptables -P OUTPUT ACCEPT

---

sudo iptables -P FORWARD DROP



showing policy settings

Explanation:

- -P sets the default Policy for a chain
- INPUT and OUTPUT accept all packets by default
- FORWARD drops all packets by default

Verification:

iptables -L -v -n

Expected output:

- Chain INPUT (policy ACCEPT)
- Chain FORWARD (policy DROP)
- Chain OUTPUT (policy ACCEPT)

Stage 3: Challenge #2 - Listing Rules

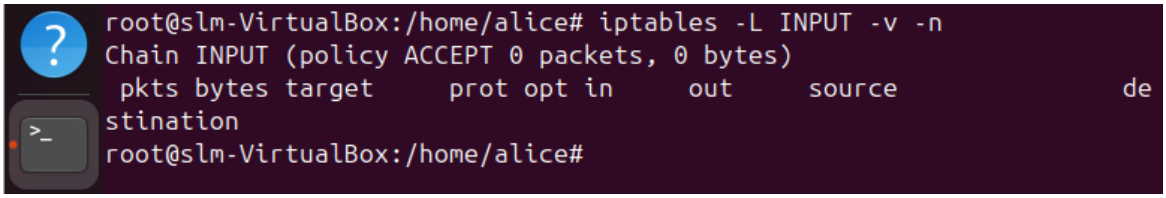
Objective: List only the filter table of the INPUT chain.

Command:

sudo iptables -L INPUT -v -n

Explanation:

- `-L INPUT` lists only INPUT chain rules
- `-v` shows verbose information (packet count, byte count)
- `-n` displays IP and port without DNS translation



Complete INPUT chain output

Expected output:

- INPUT chain is visible
- Packet/byte counts are zero (no specific rules yet)

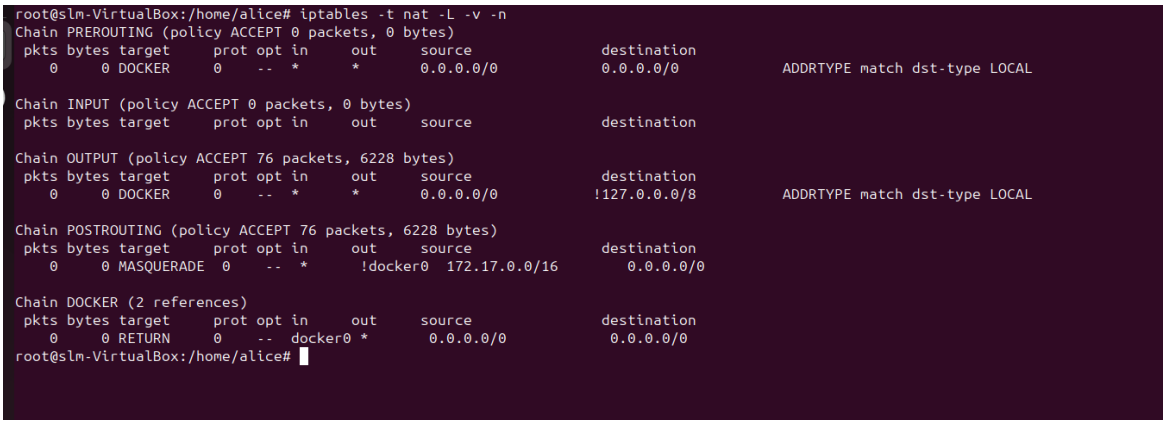
Stage 4: Challenge #3 - The NAT Table

Objective: List the NAT table.

Command:
sudo iptables -t nat -L -v -n

Explanation:

- `-t nat` selects the NAT table
- NAT table is used for network address translation
- Initially empty unless NAT was previously configured



Complete NAT table output showing PREROUTING, POSTROUTING, OUTPUT chains

Stage 5: Challenge #4 - Flushing

Objective: Flush the filter table of all chains.

Command:
sudo iptables -F

Explanation:

- -F or --flush removes all rules from all chains in the filter table
- Default Policy remains unchanged
- Essential for starting with a clean slate

Verification:
sudo iptables -L -v -n

```
root@slm-VirtualBox:/home/alice# sudo iptables -L -v -n
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain FORWARD (policy DROP 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain DOCKER (0 references)
 pkts bytes target     prot opt in     out     source               destination

Chain DOCKER-BRIDGE (0 references)
 pkts bytes target     prot opt in     out     source               destination

Chain DOCKER-CT (0 references)
 pkts bytes target     prot opt in     out     source               destination

Chain DOCKER-FORWARD (0 references)
 pkts bytes target     prot opt in     out     source               destination

Chain DOCKER-ISOLATION-STAGE-1 (0 references)
 pkts bytes target     prot opt in     out     source               destination

Chain DOCKER-ISOLATION-STAGE-2 (0 references)
 pkts bytes target     prot opt in     out     source               destination

Chain DOCKER-USER (0 references)
 pkts bytes target     prot opt in     out     source               destination
root@slm-VirtualBox:/home/alice#
```

Output showing all chains are empty

Expected output:

- All chains (INPUT, OUTPUT, FORWARD) are empty
- Packet/Byte counts are zero
- Default Policy unchanged

Stage 6: Challenge #5 - Dropping Specific Traffic

Objective: Drop all incoming packets to port 22/tcp (SSH). This should be the first rule in the chain.

Command:
sudo iptables -I INPUT 1 -p tcp --dport 22 -j DROP

Explanation:

- `-I INPUT 1` inserts rule at position 1 (first rule)
- `-p tcp` limits to TCP protocol
- `--dport 22` specifies destination port SSH
- `-j DROP` blocks packets

Verification:
sudo iptables -L INPUT -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -L INPUT -v -n --line-numbers
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 DROP       6    --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:22
root@slm-VirtualBox:/home/alice#
```

INPUT chain showing DROP rule on port 22 at line 1

Stage 7: Challenge #6 - Reset / Delete Firewall

Objective: Flush all tables and set ACCEPT policy on all chains.

WARNING: This deletes any firewall configuration.

Commands:

Flush all chains in filter table

sudo iptables -F

Flush all chains in nat table

sudo iptables -t nat -F

Flush all chains in mangle table

sudo iptables -t mangle -F

Delete all user-defined chains

sudo iptables -X

Set ACCEPT policy on all default chains

sudo iptables -P INPUT ACCEPT

---

sudo iptables -P OUTPUT ACCEPT

---

sudo iptables -P FORWARD ACCEPT

Verification:

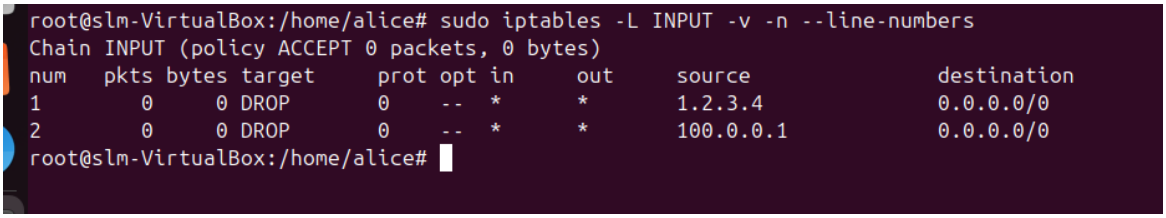
sudo iptables -L -v -n

---

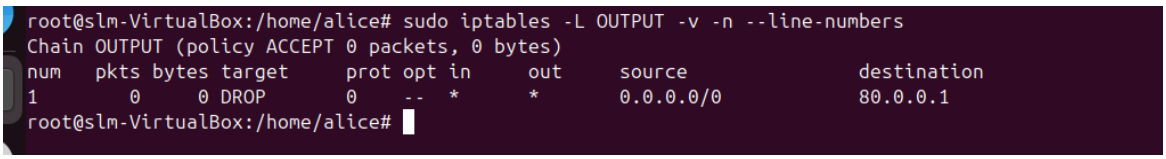
sudo iptables -t nat -L -v -n

---

sudo iptables -t mangle -L -v -n



Filter table showing all chains empty and ACCEPT policy



NAT and mangle tables showing all empty

PART 2: BASIC MATCHES

Stage 8: Challenge #1 - Filter by IP

Objective: Drop incoming packets from 100.0.0.1 and 1.2.3.4, drop outgoing packets to 80.0.0.1.

Commands:

Block incoming IPs

sudo iptables -I INPUT 1 -s 100.0.0.1 -j DROP

---

sudo iptables -I INPUT 1 -s 1.2.3.4 -j DROP

Block outgoing IP

sudo iptables -I OUTPUT 1 -d 80.0.0.1 -j DROP

Explanation:

- `-I INPUT 1` and `-I OUTPUT 1` insert rules at first position
- `-s` specifies source IP
- `-d` specifies destination IP

Verification:

sudo iptables -L INPUT -v -n --line-numbers

---

sudo iptables -L OUTPUT -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -L OUTPUT -v -n --line-numbers
Chain OUTPUT (policy ACCEPT 225K packets, 9045K bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.136.8         tcp dpt:443
2        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.136.8         tcp dpt:80
3        0     0 DROP       6    --  *      *       0.0.0.0/0            172.67.81.99         tcp dpt:443
4        0     0 DROP       6    --  *      *       0.0.0.0/0            172.67.81.99         tcp dpt:80
5        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.137.8         tcp dpt:443
6        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.137.8         tcp dpt:80
7        0     0 DROP       0    --  *      *       0.0.0.0/0            80.0.0.1
root@slm-VirtualBox:/home/alice#
```

OUTPUT chain showing blocked destination IP

```
root@slm-VirtualBox:/home/alice# sudo iptables -L INPUT -v -n --line-numbers
Chain INPUT (policy ACCEPT 284K packets, 812M bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 DROP       0    --  *      *       27.103.0.0/16        0.0.0.0/0
2        0     0 DROP       0    --  *      *       1.2.3.4              0.0.0.0/0
3        0     0 DROP       0    --  *      *       100.0.0.1            0.0.0.0/0
root@slm-VirtualBox:/home/alice#
```

INPUT chain showing blocked source IPs

Stage 9: Challenge #2 - Drop Specific Outgoing TCP

Objective: Drop all outgoing TCP packets (port 80 and 443) to www.linuxquestions.org.

Note: First resolve the IP addresses of the destination.

Example IPs: 104.24.137.8, 172.67.81.99, 104.24.136.8

Commands:

For 104.24.137.8

sudo iptables -I OUTPUT 1 -p tcp -d 104.24.137.8 --dport 80 -j DROP

---

sudo iptables -I OUTPUT 1 -p tcp -d 104.24.137.8 --dport 443 -j DROP

For 172.67.81.99

sudo iptables -I OUTPUT 1 -p tcp -d 172.67.81.99 --dport 80 -j DROP

---

sudo iptables -I OUTPUT 1 -p tcp -d 172.67.81.99 --dport 443 -j DROP

For 104.24.136.8

sudo iptables -I OUTPUT 1 -p tcp -d 104.24.136.8 --dport 80 -j DROP

---

sudo iptables -I OUTPUT 1 -p tcp -d 104.24.136.8 --dport 443 -j DROP

Verification:

sudo iptables -L OUTPUT -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -L OUTPUT -v -n --line-numbers
Chain OUTPUT (policy ACCEPT 225K packets, 9045K bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.136.8         tcp dpt:443
2        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.136.8         tcp dpt:80
3        0     0 DROP       6    --  *      *       0.0.0.0/0            172.67.81.99         tcp dpt:443
4        0     0 DROP       6    --  *      *       0.0.0.0/0            172.67.81.99         tcp dpt:80
5        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.137.8         tcp dpt:443
6        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.137.8         tcp dpt:80
7        0     0 DROP       0    --  *      *       0.0.0.0/0            80.0.0.1
root@slm-VirtualBox:/home/alice#
```

OUTPUT chain showing all TCP blocks for specified IPs

Stage 10: Challenge #3 - Drop Routed Traffic

Objective: Drop all routed outgoing packets (TCP port 80 and 443) to www.linuxquestions.org.

Commands:

For 104.24.137.8

sudo iptables -I FORWARD 1 -p tcp -d 104.24.137.8 --dport 80 -j DROP

---

sudo iptables -I FORWARD 1 -p tcp -d 104.24.137.8 --dport 443 -j DROP

For 172.67.81.99

sudo iptables -I FORWARD 1 -p tcp -d 172.67.81.99 --dport 80 -j DROP

---

sudo iptables -I FORWARD 1 -p tcp -d 172.67.81.99 --dport 443 -j DROP

For 104.24.136.8

sudo iptables -I FORWARD 1 -p tcp -d 104.24.136.8 --dport 80 -j DROP

---

sudo iptables -I FORWARD 1 -p tcp -d 104.24.136.8 --dport 443 -j DROP

Explanation:

- Uses FORWARD chain for routed traffic
- Blocks traffic passing through the router from LAN to WAN

Verification:

sudo iptables -L FORWARD -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -L FORWARD -v -n --line-numbers
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.136.8         tcp dpt:443
2        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.136.8         tcp dpt:80
3        0     0 DROP       6    --  *      *       0.0.0.0/0            172.67.81.99         tcp dpt:443
4        0     0 DROP       6    --  *      *       0.0.0.0/0            172.67.81.99         tcp dpt:80
5        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.137.8         tcp dpt:443
6        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.137.8         tcp dpt:80
root@slm-VirtualBox:/home/alice#
```

FORWARD chain showing TCP blocks for specified IPs

Stage 11: Challenge #4 - Drop Subnet

Objective: Drop all incoming packets from network 27.103.0.0/16 (first rule in chain).

Command:

sudo iptables -I INPUT 1 -s 27.103.0.0/16 -j DROP

Explanation:

- `-s 27.103.0.0/16` specifies source network range
- Netmask 255.255.0.0 covers entire /16 subnet

Verification:

sudo iptables -L INPUT -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -L INPUT -v -n --line-numbers
Chain INPUT (policy ACCEPT 284K packets, 812M bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 DROP       0    --  *      *       27.103.0.0/16        0.0.0.0/0
2        0     0 DROP       0    --  *      *       1.2.3.4              0.0.0.0/0
3        0     0 DROP       0    --  *      *       100.0.0.1            0.0.0.0/0
root@slm-VirtualBox:/home/alice#
```

INPUT chain showing DROP rule for subnet 27.103.0.0/16 at line 1

Stage 12: Challenge #5 - Enforce DNS Server

Objective: Drop UDP packets to port 53 (DNS) if destined to any IP other than 8.8.8.8.

Command:

sudo iptables -I FORWARD 1 -p udp --dport 53 ! -d 8.8.8.8 -j DROP

Explanation:

- `-p udp --dport 53` matches DNS traffic
- `! -d 8.8.8.8` means NOT destination 8.8.8.8 (negation)
- Forces LAN users to use only authorized DNS server

Verification:

sudo iptables -L FORWARD -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -L FORWARD -v -n --line-numbers
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 DROP       17   --  *      *       0.0.0.0/0           !8.8.8.8             udp dpt:53
2        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.136.8        tcp dpt:443
3        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.136.8        tcp dpt:80
4        0     0 DROP       6    --  *      *       0.0.0.0/0            172.67.81.99        tcp dpt:443
5        0     0 DROP       6    --  *      *       0.0.0.0/0            172.67.81.99        tcp dpt:80
6        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.137.8        tcp dpt:443
7        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.137.8        tcp dpt:80
root@slm-VirtualBox:/home/alice#
```

FORWARD chain showing DNS enforcement rule

Stage 13: Challenge #6 - Allow Loopback

Objective: Allow all traffic on the loopback (lo) interface.

Commands:

sudo iptables -I INPUT 1 -i lo -j ACCEPT

---

sudo iptables -I OUTPUT 1 -o lo -j ACCEPT

Explanation:

- -i lo specifies incoming loopback interface
- -o lo specifies outgoing loopback interface
- Essential for internal system services

Verification:

sudo iptables -L INPUT -v -n --line-numbers

---

sudo iptables -L OUTPUT -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -L INPUT -v -n --line-numbers
Chain INPUT (policy ACCEPT 284K packets, 812M bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     0    --  lo     *       0.0.0.0/0            0.0.0.0/0
2        0     0 DROP       0    --  *      *       27.103.0.0/16        0.0.0.0/0
3        0     0 DROP       0    --  *      *       1.2.3.4              0.0.0.0/0
4        0     0 DROP       0    --  *      *       100.0.0.1            0.0.0.0/0
root@slm-VirtualBox:/home/alice#
```

INPUT chain showing loopback ACCEPT rule

```
root@slm-VirtualBox:/home/alice# sudo iptables -L OUTPUT -v -n --line-numbers
Chain OUTPUT (policy ACCEPT 225K packets, 9047K bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     0    --  *      lo      0.0.0.0/0            0.0.0.0/0
2        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.136.8         tcp dpt:443
3        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.136.8         tcp dpt:80
4        0     0 DROP       6    --  *      *       0.0.0.0/0            172.67.81.99         tcp dpt:443
5        0     0 DROP       6    --  *      *       0.0.0.0/0            172.67.81.99         tcp dpt:80
6        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.137.8         tcp dpt:443
7        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.137.8         tcp dpt:80
8        0     0 DROP       0    --  *      *       0.0.0.0/0            80.0.0.1
root@slm-VirtualBox:/home/alice#
```

OUTPUT chain showing loopback ACCEPT rule

Stage 14: Challenge #7 - Interface Specific SSH

Objective: Allow routed incoming SSH (tcp/22) only from LAN (enp0s8). Drop SSH from WAN (enp0s3).

Commands:

Allow SSH from LAN

sudo iptables -I FORWARD 1 -p tcp -i enp0s8 --dport 22 -j ACCEPT

Block SSH from WAN

sudo iptables -I FORWARD 2 -p tcp -i enp0s3 --dport 22 -j DROP

Explanation:

- `-i enp0s8` specifies LAN interface
- `-i enp0s3` specifies WAN interface
- Rule order matters: ACCEPT first, then DROP

Verification:

sudo iptables -L FORWARD -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -L FORWARD -v -n --line-numbers
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     6    --  enp0s8 *       0.0.0.0/0            0.0.0.0/0           tcp dpt:22
2        0     0 DROP       6    --  enp0s3 *       0.0.0.0/0            0.0.0.0/0           tcp dpt:22
3        0     0 DROP       17   --  *      *       0.0.0.0/0           !8.8.8.8             udp dpt:53
4        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.136.8        tcp dpt:443
5        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.136.8        tcp dpt:80
6        0     0 DROP       6    --  *      *       0.0.0.0/0            172.67.81.99        tcp dpt:443
7        0     0 DROP       6    --  *      *       0.0.0.0/0            172.67.81.99        tcp dpt:80
8        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.137.8        tcp dpt:443
9        0     0 DROP       6    --  *      *       0.0.0.0/0            104.24.137.8        tcp dpt:80
root@slm-VirtualBox:/home/alice#
```

FORWARD chain shows SSH allowed from LAN and blocked from WAN

PART 3: ADVANCED MATCHES

Stage 15: Challenge #1 - Help Commands

Objective: List help for time, mac, and limit matches.

Commands:

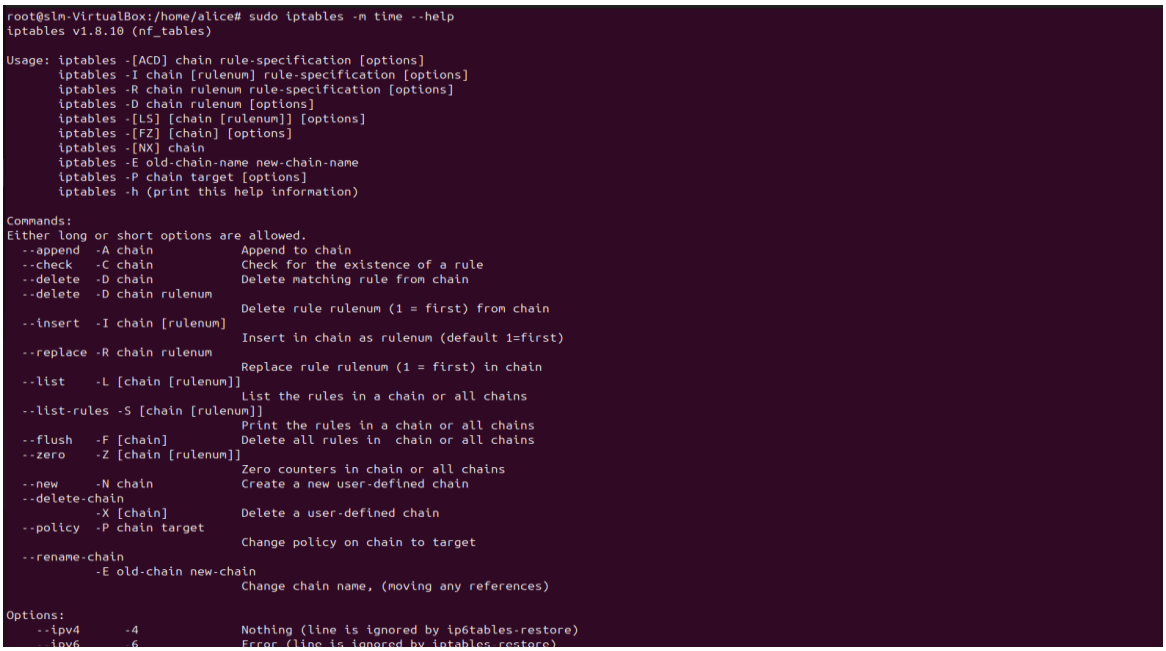
sudo iptables -m time --help

---

sudo iptables -m mac --help

---

sudo iptables -m limit –help



```
root@slm-VirtualBox:/home/alice# sudo iptables -m time --help
iptables v1.8.10 (nf_tables)

Usage: iptables -[ACD] chain rule-specification [options]
       iptables -I chain [rulenum] rule-specification [options]
       iptables -R chain rulenum rule-specification [options]
       iptables -D chain rulenum [options]
       iptables -[LS] [chain [rulenum]] [options]
       iptables -[FZ] [chain] [options]
       iptables -[NX] chain
       iptables -E old-chain-name new-chain-name
       iptables -P chain target [options]
       iptables -h (print this help information)

Commands:
Either long or short options are allowed.
  --append  -A chain            Append to chain
  --check   -C chain            Check for the existence of a rule
  --delete  -D chain            Delete matching rule from chain
  --delete  -D chain rulenum
                                Delete rule rulenum (1 = first) from chain
  --insert  -I chain [rulenum]
                                Insert in chain as rulenum (default 1=first)
  --replace -R chain rulenum
                                Replace rule rulenum (1 = first) in chain
  --list    -L [chain [rulenum]]
                                List the rules in a chain or all chains
  --list-rules -S [chain [rulenum]]
                                Print the rules in a chain or all chains
  --flush   -F [chain]          Delete all rules in  chain or all chains
  --zero    -Z [chain [rulenum]]
                                Zero counters in chain or all chains
  --new     -N chain            Create a new user-defined chain
  --delete-chain
            -X [chain]          Delete a user-defined chain
  --policy  -P chain target
                                Change policy on chain to target
  --rename-chain
            -E old-chain new-chain
                                Change chain name, (moving any references)

Options:
    --ipv4      -4              Nothing (line is ignored by ip6tables-restore)
    --ipv6      -6              Error (line is ignored by iptables-restore)
```

Output of time match help

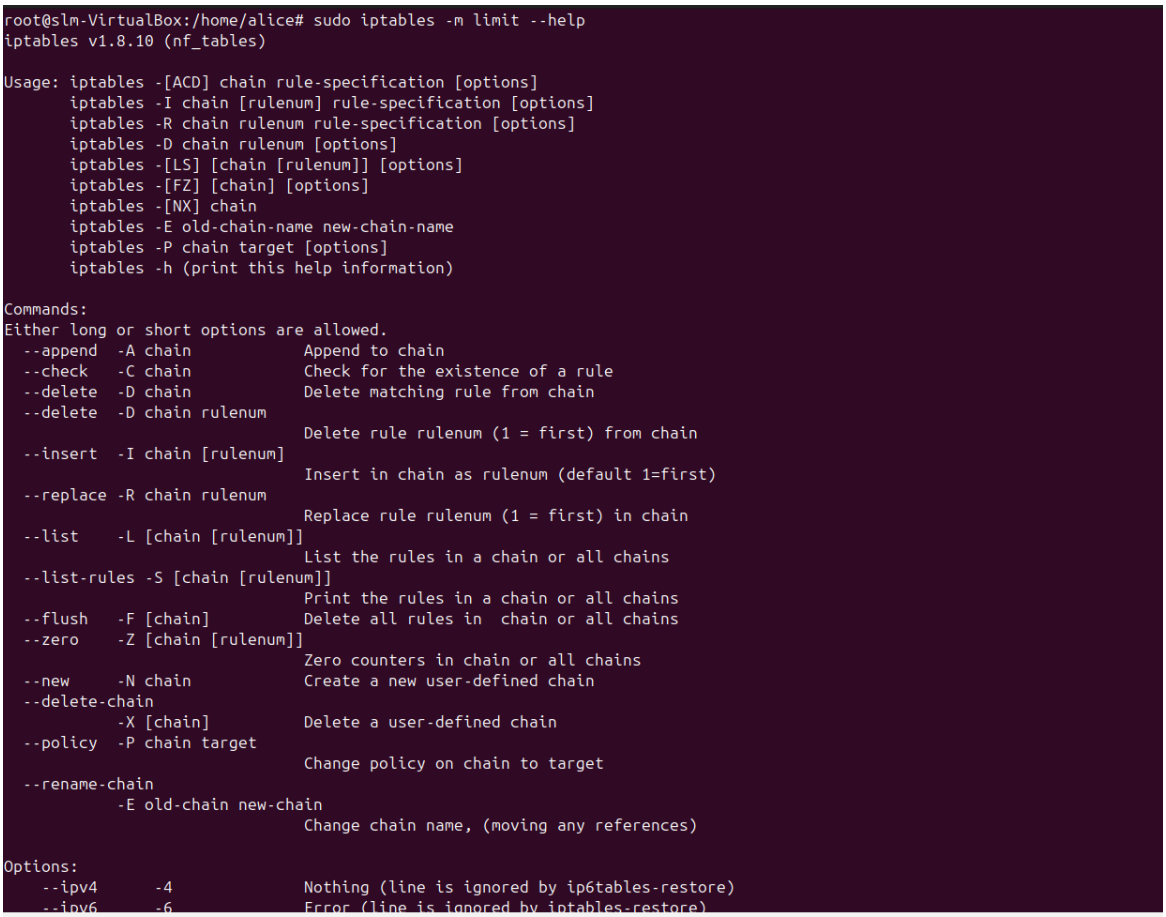
```
root@slm-VirtualBox:/home/alice# sudo iptables -m mac --help
iptables v1.8.10 (nf_tables)

Usage: iptables -[ACD] chain rule-specification [options]
       iptables -I chain [rulenum] rule-specification [options]
       iptables -R chain rulenum rule-specification [options]
       iptables -D chain rulenum [options]
       iptables -[LS] [chain [rulenum]] [options]
       iptables -[FZ] [chain] [options]
       iptables -[NX] chain
       iptables -E old-chain-name new-chain-name
       iptables -P chain target [options]
       iptables -h (print this help information)

Commands:
Either long or short options are allowed.
  --append  -A chain            Append to chain
  --check   -C chain            Check for the existence of a rule
  --delete  -D chain            Delete matching rule from chain
  --delete  -D chain rulenum
                                Delete rule rulenum (1 = first) from chain
  --insert  -I chain [rulenum]
                                Insert in chain as rulenum (default 1=first)
  --replace -R chain rulenum
                                Replace rule rulenum (1 = first) in chain
  --list    -L [chain [rulenum]]
                                List the rules in a chain or all chains
  --list-rules -S [chain [rulenum]]
                                Print the rules in a chain or all chains
  --flush   -F [chain]          Delete all rules in  chain or all chains
  --zero    -Z [chain [rulenum]]
                                Zero counters in chain or all chains
  --new     -N chain            Create a new user-defined chain
  --delete-chain
            -X [chain]          Delete a user-defined chain
  --policy  -P chain target
                                Change policy on chain to target
  --rename-chain
            -E old-chain new-chain
                                Change chain name, (moving any references)

Options:
    --ipv4      -4              Nothing (line is ignored by ip6tables-restore)
    --ipv6      -6              Error (line is ignored by iptables-restore)
```

Output of mac match help



Output of limit match help

Stage 16: Challenge #2 - Stateful Firewall (Basic)

Objective: Create a basic stateful firewall for a laptop. All outgoing traffic allowed, only return incoming traffic permitted.

Create script:
sudo nano stateful_fw.sh

Script content:
#!/bin/bash

Flush all chains

iptables -F

---

iptables -X

Set default policies

iptables -P INPUT DROP

---

iptables -P OUTPUT ACCEPT

---

iptables -P FORWARD DROP

Allow loopback traffic

iptables -A INPUT -i lo -j ACCEPT

Allow established and related connections

iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

Execute script:

sudo chmod +x stateful_fw.sh

---

sudo ./stateful_fw.sh

Verification:

sudo iptables -L INPUT -v -n --line-numbers

---

sudo iptables -L OUTPUT -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -L INPUT -v -n --line-numbers
Chain INPUT (policy DROP 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     0    --  lo     *       0.0.0.0/0            0.0.0.0/0
2        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           state RELATED,ESTABLISHED
root@slm-VirtualBox:/home/alice#
```

INPUT chain showing default DROP and ESTABLISHED,RELATED ACCEPT

```
root@slm-VirtualBox:/home/alice# sudo iptables -L OUTPUT -v -n --line-numbers
Chain OUTPUT (policy ACCEPT 9 packets, 652 bytes)
num   pkts bytes target     prot opt in     out     source               destination
root@slm-VirtualBox:/home/alice#
```

OUTPUT chain showing default ACCEPT

Stage 17: Challenge #3 - Stateful Firewall (Complete)

Objective: Enhance previous firewall with loopback, INVALID drop, and flush at start.

Create script:
sudo nano stateful_fw_complete.sh

Script content:
#!/bin/bash

Flush all tables and delete custom chains

iptables -F

---

iptables -X

Set default policies

iptables -P INPUT DROP

---

iptables -P OUTPUT ACCEPT

---

iptables -P FORWARD DROP

Allow loopback traffic

iptables -A INPUT -i lo -j ACCEPT

Drop INVALID packets explicitly

iptables -A INPUT -m state --state INVALID -j DROP

Allow established and related connections

iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

Execute script:
sudo chmod +x stateful_fw_complete.sh

---

sudo ./stateful_fw_complete.sh

Verification:
sudo iptables -L INPUT -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -L INPUT -v -n --line-numbers
Chain INPUT (policy DROP 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     0    --  lo     *       0.0.0.0/0            0.0.0.0/0
2        0     0 DROP       0    --  *      *       0.0.0.0/0            0.0.0.0/0           state INVALID
3        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           state RELATED,ESTABLISHED
root@slm-VirtualBox:/home/alice#
```

INPUT chain showing loopback, INVALID drop, and stateful rules

Stage 18: Challenge #4 - Trusted Source SSH

Objective: Allow incoming SSH connections (tcp/22) only from trusted IP 100.0.0.1.

Command:

sudo iptables -A INPUT -p tcp -s 100.0.0.1 --dport 22 -m state --state NEW,ESTABLISHED -j ACCEPT

Explanation:

- `-s 100.0.0.1` limits SSH to trusted source only
- `--state NEW,ESTABLISHED` allows new connections and established ones

Verification:

sudo iptables -L INPUT -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -L INPUT -v -n --line-numbers
Chain INPUT (policy DROP 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     0    --  lo     *       0.0.0.0/0            0.0.0.0/0
2        0     0 DROP       0    --  *      *       0.0.0.0/0            0.0.0.0/0           state INVALID
3        9  1701 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           state RELATED,ESTABLISHED
4        0     0 ACCEPT     6    --  *      *       100.0.0.1            0.0.0.0/0           tcp dpt:22 state NEW,ESTABLISHED
root@slm-VirtualBox:/home/alice#
```

INPUT chain showing SSH allowed only from 100.0.0.1

Stage 19: Challenge #5 - MAC Filtering (Router Only)

Objective: Allow communication only with router MAC b4:6d:83:77:85:f5.

Commands:

sudo iptables -I INPUT 1 -m mac --mac-source b4:6d:83:77:85:f5 -j ACCEPT

---

sudo iptables -I INPUT 2 -j DROP

Explanation:

- First rule accepts traffic from router MAC
- Second rule drops all other traffic

Verification:

sudo iptables -L INPUT -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -L INPUT -v -n --line-numbers
Chain INPUT (policy DROP 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           MAC b4:6d:83:77:85:f5
2        0     0 DROP       0    --  *      *       0.0.0.0/0            0.0.0.0/0
3        0     0 ACCEPT     0    --  lo     *       0.0.0.0/0            0.0.0.0/0
4        0     0 DROP       0    --  *      *       0.0.0.0/0            0.0.0.0/0           state INVALID
5       18  3402 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           state RELATED,ESTABLISHED
6        0     0 ACCEPT     6    --  *      *       100.0.0.1            0.0.0.0/0           tcp dpt:22 state NEW,ESTABLISHED
root@slm-VirtualBox:/home/alice#
```

INPUT chain showing MAC filter rules

Stage 20: Challenge #6 - MAC Whitelist Script

Objective: Allow only 5 trusted hosts with MACs ending in f1 through f5.

Create script:

sudo nano mac_whitelist.sh

Script content:

#!/bin/bash

Flush INPUT chain

iptables -F

Allow loopback

iptables -A INPUT -i lo -j ACCEPT

Allow 5 trusted MACs

for mac in b4:6d:83:77:85:f1 b4:6d:83:77:85:f2 b4:6d:83:77:85:f3 b4:6d:83:77:85:f4 b4:6d:83:77:85:f5
do
iptables -A INPUT -m mac --mac-source $mac -j ACCEPT
done

Drop all other traffic

iptables -A INPUT -j DROP

Execute script:

sudo chmod +x mac_whitelist.sh

sudo ./mac_whitelist.sh

Verification:

sudo iptables -L INPUT -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -L INPUT -v -n --line-numbers
Chain INPUT (policy DROP 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     0    --  lo     *       0.0.0.0/0            0.0.0.0/0
2        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           MAC b4:6d:83:77:85:f1
3        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           MAC b4:6d:83:77:85:f2
4        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           MAC b4:6d:83:77:85:f3
5        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           MAC b4:6d:83:77:85:f4
6        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           MAC b4:6d:83:77:85:f5
7        0     0 DROP       0    --  *      *       0.0.0.0/0            0.0.0.0/0
root@slm-VirtualBox:/home/alice#
```

INPUT chain showing 5 MAC ACCEPT rules followed by DROP

Stage 21: Challenge #7 - Time-Based Web Access

Objective: Allow outgoing web traffic (TCP 80/443) only between 10:00 and 18:00 UTC.

Commands:

sudo iptables -A OUTPUT -p tcp --dport 80 -m time --timestart 10:00 --timestop 18:00 -- kerneltz -j ACCEPT

sudo iptables -A OUTPUT -p tcp --dport 443 -m time --timestart 10:00 --timestop 18:00 --kerneltz -j ACCEPT

Explanation:

- `-m time` enables time matching
- `--timestart` and `--timestop` define time window
- `--kerneltz` uses system timezone

Verification:

sudo iptables -L OUTPUT -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -L OUTPUT -v -n --line-numbers
Chain OUTPUT (policy ACCEPT 103 packets, 6742 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     6    --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:80 TIME from 10:00:00 to 18:00:00
2        0     0 ACCEPT     6    --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:443 TIME from 10:00:00 to 18:00:00
root@slm-VirtualBox:/home/alice#
```

OUTPUT chain showing time-based web access rules

Stage 22: Challenge #8 - Weekend Access Only

Objective: Allow web traffic only on weekends (Sat, Sun) between 10:00 and 18:00 UTC.

Commands:

sudo iptables -A OUTPUT -p tcp --dport 80 -m time --timestart 10:00 --timestop 18:00 --weekdays Sat,Sun --kerneltz -j ACCEPT

---

sudo iptables -A OUTPUT -p tcp --dport 443 -m time --timestart 10:00 --timestop 18:00 --weekdays Sat,Sun --kerneltz -j ACCEPT

Explanation:

- `--weekdays Sat,Sun` restricts to weekend only
- Combines time and day restrictions

Verification:

sudo iptables -L OUTPUT -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -L OUTPUT -v -n --line-numbers
Chain OUTPUT (policy ACCEPT 113 packets, 7368 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     6    --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:80 TIME from 10:00:00 to 18:00:00
2        0     0 ACCEPT     6    --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:443 TIME from 10:00:00 to 18:00:00
3        0     0 ACCEPT     6    --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:80 TIME from 10:00:00 to 18:00:00 on Sat,Sun
4        0     0 ACCEPT     6    --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:443 TIME from 10:00:00 to 18:00:00 on Sat,Sun
root@slm-VirtualBox:/home/alice#
```

OUTPUT chain showing weekend time-based rules

Stage 23: Challenge #9 - Limit Ping

Objective: Permit only 2 incoming ICMP echo-request (ping) packets per second.

Command:

sudo iptables -A INPUT -p icmp --icmp-type echo-request -m limit --limit 2/sec --limit-burst 2 -j ACCEPT

Explanation:

- `-m limit` enables rate limiting
- `--limit 2/sec` allows 2 packets per second maximum
- `--limit-burst 2` sets initial burst allowance
- Prevents ping flooding

Verification:

sudo iptables -L INPUT -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -L INPUT -v -n --line-numbers
Chain INPUT (policy DROP 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     0    --  lo     *       0.0.0.0/0            0.0.0.0/0
2        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           MAC b4:6d:83:77:85:f1
3        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           MAC b4:6d:83:77:85:f2
4        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           MAC b4:6d:83:77:85:f3
5        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           MAC b4:6d:83:77:85:f4
6        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           MAC b4:6d:83:77:85:f5
7      104  7988 DROP       0    --  *      *       0.0.0.0/0            0.0.0.0/0
8        0     0 ACCEPT     1    --  *      *       0.0.0.0/0            0.0.0.0/0           icmptype 8 limit: avg 2/sec burst 2
root@slm-VirtualBox:/home/alice#
```

INPUT chain showing ICMP rate limit rule

Stage 24: Challenge #10 - Limit TCP Connections

Objective: Permit only 10 NEW TCP connections from the same IP address.

Command:

sudo iptables -A INPUT -p tcp --syn -m connlimit --connlimit-above 10 -j REJECT

Explanation:

- `--syn` matches TCP connection initiation packets
- `-m connlimit` enables connection limiting
- `--connlimit-above 10` rejects if more than 10 concurrent connections
- Prevents SYN flood attacks

Verification:

sudo iptables -L INPUT -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -L INPUT -v -n --line-numbers
Chain INPUT (policy DROP 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     0    --  lo     *       0.0.0.0/0            0.0.0.0/0
2        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           MAC b4:6d:83:77:85:f1
3        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           MAC b4:6d:83:77:85:f2
4        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           MAC b4:6d:83:77:85:f3
5        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           MAC b4:6d:83:77:85:f4
6        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0           MAC b4:6d:83:77:85:f5
7      113  8952 DROP       0    --  *      *       0.0.0.0/0            0.0.0.0/0
8        0     0 ACCEPT     1    --  *      *       0.0.0.0/0            0.0.0.0/0           icmptype 8 limit: avg 2/sec burst 2
9        0     0 REJECT     6    --  *      *       0.0.0.0/0            0.0.0.0/0           tcp flags:0x17/0x02 #conn src/32 > 10 reject-with icmp-port-unreachable
root@slm-VirtualBox:/home/alice#
```

INPUT chain showing connection limit rule

PART 4: NAT & PORT FORWARDING

Stage 25: Challenge #1 - Basic NAT Configuration

Objective: Configure NAT for LAN network (10.0.2.0/24).

Commands:

Flush NAT table

sudo iptables -t nat -F

Enable IP Forwarding

sudo sysctl -w net.ipv4.ip_forward=1

Apply SNAT for entire subnet (assuming public IP is 80.0.0.1)

sudo iptables -t nat -A POSTROUTING -s 10.0.2.0/24 -o enp0s3 -j SNAT --to-source 80.0.0.1

For permanent IP forwarding, edit /etc/sysctl.conf:
net.ipv4.ip_forward=1

Then run:
sudo sysctl -p

BONUS - For dynamic IP, use MASQUERADE:
sudo iptables -t nat -A POSTROUTING -s 10.0.2.0/24 -o enp0s3 -j MASQUERADE

Verification:
sudo iptables -t nat -L -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -t nat -L -v -n --line-numbers
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 93 packets, 7580 bytes)
num   pkts bytes target     prot opt in     out     source               destination

Chain POSTROUTING (policy ACCEPT 93 packets, 7580 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 SNAT       0    --  *      enp0s3  10.0.2.0/24         0.0.0.0/0            to:80.0.0.1
2        0     0 MASQUERADE  0    --  *      enp0s3  10.0.2.0/24          0.0.0.0/0

Chain DOCKER (0 references)
num   pkts bytes target     prot opt in     out     source               destination
root@slm-VirtualBox:/home/alice#
```

NAT table showing POSTROUTING rule for subnet 10.0.2.0/24

Explanation:

- SNAT translates LAN private IPs to public IP
- MASQUERADE automatically adjusts if public IP changes
- Essential for LAN internet access through router

Stage 26: Challenge #2 - Port Forwarding (DNAT)

Objective: Forward external traffic on port 80 to internal web server at 192.168.0.20:80.

Commands:

Basic port forwarding (port 80)

sudo iptables -t nat -A PREROUTING -i enp0s3 -p tcp --dport 80 -j DNAT --to-destination 192.168.0.20:80

Allow forwarding in FORWARD chain

sudo iptables -A FORWARD -p tcp -d 192.168.0.20 --dport 80 -j ACCEPT

VARIANT - Redirect external port 8080 to internal server port 80:
sudo iptables -t nat -A PREROUTING -i enp0s3 -p tcp --dport 8080 -j DNAT --to-destination 192.168.0.20:80

---

sudo iptables -A FORWARD -p tcp -d 192.168.0.20 --dport 80 -j ACCEPT

Verification:
sudo iptables -t nat -L -v -n --line-numbers

---

sudo iptables -L FORWARD -v -n --line-numbers

```
root@slm-VirtualBox:/home/alice# sudo iptables -t nat -L -v -n --line-numbers
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 DNAT       6    --  enp0s3 *       0.0.0.0/0            0.0.0.0/0           tcp dpt:80 to:192.168.0.20:80
2        0     0 DNAT       6    --  enp0s3 *       0.0.0.0/0            0.0.0.0/0           tcp dpt:8080 to:192.168.0.20:80

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 96 packets, 7812 bytes)
num   pkts bytes target     prot opt in     out     source               destination

Chain POSTROUTING (policy ACCEPT 93 packets, 7580 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        3   232 SNAT       0    --  *      enp0s3  10.0.2.0/24         0.0.0.0/0           to:80.0.0.1
2        0     0 MASQUERADE  0    --  *      enp0s3  10.0.2.0/24          0.0.0.0/0

Chain DOCKER (0 references)
num   pkts bytes target     prot opt in     out     source               destination
root@slm-VirtualBox:/home/alice#
```

NAT table showing DNAT rules for ports 80 and 8080

```
root@slm-VirtualBox:/home/alice# sudo iptables -L FORWARD -v -n --line-numbers
Chain FORWARD (policy DROP 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     6    --  *      *       0.0.0.0/0            192.168.0.20         tcp dpt:80
root@slm-VirtualBox:/home/alice#
```

FORWARD chain showing allowed traffic to internal server

Explanation:

- PREROUTING chain changes destination before routing decision
- External users access internal web server via router's public IP
- Router forwards traffic to internal server transparently

Lab Completion Summary

This lab covered the complete configuration of Linux packet filtering firewalls using iptables command-line interface:

Part 1: Basic iptables commands (policies, listing, flushing, basic rules)

Part 2: Basic matches (IP filtering, subnet blocking, interface-specific rules, DNS enforcement)

Part 3: Advanced matches (stateful firewall, MAC filtering, time-based access, rate limiting)

Part 4: NAT and port forwarding (SNAT, MASQUERADE, DNAT)

All challenges have been completed with detailed commands, explanations, and screenshot placeholders for documentation.